

A New Efficient Algorithm for Computing the Longest Common Subsequence

Costas S. Iliopoulos · M. Sohel Rahman

Published online: 25 January 2008
© Springer Science+Business Media, LLC 2008

Abstract The *Longest Common Subsequence* (LCS) problem is a classic and well-studied problem in computer science. The LCS problem is a common task in DNA sequence analysis with many applications to genetics and molecular biology. In this paper, we present a new and efficient algorithm for solving the LCS problem for two strings. Our algorithm runs in $O(\mathcal{R} \log \log n + n)$ time, where $\mathcal{R}$ is the total number of ordered pairs of positions at which the two strings match.

Keywords Longest common subsequence · Algorithms · Strings

1 Introduction

The *Longest Common Subsequence* (LCS) problem is a classic and well-studied problem in computer science with extensive applications in diverse areas ranging from spelling error corrections to molecular biology. A subsequence of a string is obtained by deleting zero or more symbols of that string. The longest common subsequence problem for two strings, is to find a common subsequence in both strings, having maximum possible length. More formally, suppose we are given

Preliminary version appeared in [24].

C.S. Iliopoulos is supported by EPSRC and Royal Society grants.

M.S. Rahman is supported by the Commonwealth Scholarship Commission in the UK under the Commonwealth Scholarship and Fellowship Plan (CSFP) and is on Leave from Department of CSE, BUET, Dhaka-1000, Bangladesh.

C.S. Iliopoulos · M.S. Rahman (✉)

Algorithm Design Group, Department of Computer Science, King's College London, Strand, London WC2R 2LS, England, UK

e-mail: sohel@dcs.kcl.ac.uk

C.S. Iliopoulos

e-mail: csi@dcs.kcl.ac.uk

two strings $X[1..n] = X[1]X[2] \cdots X[n]$ and $Y[1..n] = Y[1]Y[2] \cdots Y[n]$. A subsequence $S[1..r] = S[1]S[2] \cdots S[r]$, $0 \leq r \leq n$ of X is obtained by deleting $n - r$ symbols from X . A common subsequence of two strings X and Y , denoted $cs(X, Y)$, is a subsequence common to both X and Y . The longest common subsequence of X and Y , denoted $lcs(X, Y)$ or $LCS(X, Y)$, is a common subsequence of maximum length. We denote the length of $lcs(X, Y)$ by $r(X, Y)$. In this paper, we assume that the two given strings are of equal length. But our results can be easily extended to handle two strings of different length.

Problem “LCS” (LCS Problem for 2 Strings) *Given strings X and Y , compute the Longest Common Subsequence of X and Y .*

The longest common subsequence problem for k strings ($k > 2$) was first shown to be NP-hard [18] and later proved to be hard to be approximated [15]. The restricted but, probably, the more studied problem that deals with two strings has been studied extensively [9–11, 19, 21, 22, 27]. The classic dynamic programming solution to LCS problem, invented by Wagner and Fischer [27], has $O(n^2)$ worst case running time. Masek and Paterson [19] improved this algorithm using the “Four-Russians” technique [1] to reduce the worst case running time to $O(n^2/\log n)$.¹ Since then, not much improvement in terms of n can be found in the literature. However, several algorithms exist with complexities depending on other parameters. For example, Myers in [21] and Nakatsu et al. in [22] presented an $O(nD)$ algorithm, where the parameter D is the simple Levenshtein distance between the two given strings [16]. Another interesting and perhaps more relevant parameter for this problem is $\mathcal{R}$, which is the total number of ordered pairs of positions at which the two strings match. More formally, we say a pair (i, j) , $1 \leq i, j \leq n$, defines a match, if $X[i] = Y[j]$. The set of all matches, M , is defined as follows:

$$M = \{(i, j) \mid X[i] = Y[j], 1 \leq i, j \leq n\}.$$

Observe that $|M| = \mathcal{R}$. Hunt and Szymanski [11] presented an algorithm to solve Problem LCS in $O((\mathcal{R} + n) \log n)$ time. They also cited applications, where $\mathcal{R} \sim n$ and thereby claimed that for these applications the algorithm would run in $O(n \log n)$ time. For a comprehensive comparison of the well-known algorithms for LCS problem and study of their behavior in various application environments, the readers are referred to [5].

In this paper, we revisit the much studied LCS problem for two strings and present new algorithms using some novel ideas and interesting observations. Our main result is an $O(\mathcal{R} \log \log n + n)$ time algorithm for Problem LCS.² Notably, an

¹Employing different techniques, the same worst case bound was achieved in [7]. In particular, for most texts, the achieved time complexity in [7] is $O(hn^2/\log n)$, where $h \leq 1$ is the entropy of the text.

²Very recently in [12, 14], we have presented another algorithm to solve the LCS problem achieving the same running time using a new data structure from [6]. This algorithm was then used to devise an efficient algorithm to solve the Constrained LCS problem, which is a relatively new and interesting variant of the classic LCS problem.

$O(\mathcal{R} \log \log n)$ time algorithm for LCS was also reported in [20]; but their running time excludes a costly preprocessing time of $O(n^2 \log n)$.

The rest of the paper is organized as follows. In Sect. 2, we present an $O(n^2 + \mathcal{R} \log \log n)$ time algorithm, namely LCS-I, to solve Problem LCS, which is an easy extension of the algorithms and techniques of [13, 23]. LCS-I provides the base of our new improved algorithm, LCS-II, described in Sect. 3. Using some novel techniques, we achieve $O(\mathcal{R} \log \log n + n)$ running time for LCS-II. Finally, we briefly conclude in Sect. 4.

2 A New Algorithm

In this section, we present Algorithm LCS-I, which works in $O(n^2 + \mathcal{R} \log \log n)$ time. Note that, LCS-I is an easy extension of the algorithms presented in [13, 23]³ and the main contribution of this paper is an improved algorithm, namely LCS-II, with $O(\mathcal{R} \log \log n + n)$ running time, to be presented in Sect. 3.

From the definition of LCS, it is clear that, if $(i, j) \in M$, then we can calculate $\mathcal{T}[i, j]$, $1 \leq i, j \leq n$ by employing the following equation [13, 23]:

$$\mathcal{T}[i, j] = \begin{cases} \text{Undefined} & \text{if } (i, j) \notin M, \\ 1 & \text{if } (i, j) \in M \text{ and} \\ & (i', j') \notin M, i' < i, j' < j, \\ \max_{\substack{1 \leq \ell_i < i \\ 1 \leq \ell_j < j \\ (\ell_i, \ell_j) \in M}} \{(\mathcal{T}[\ell_i, \ell_j])\} + 1 & \text{Otherwise.} \end{cases} \quad (1)$$

Here, we have used the tabular notion $\mathcal{T}[i, j]$ to denote $r(X[1..i], Y[1..j])$, when $(i, j) \in M$.⁴ From (1), it follows that, only the entries $\mathcal{T}[i, j]$ such that $(i, j) \in M$ are useful. Therefore, we can ignore all $\mathcal{T}[i, j]$ with $(i, j) \notin M$ from the calculation. In order to do that, we need a preprocessing step to construct the set M sorted according to the order they would be considered in the algorithm (we consider a row by row operation, i.e. all the matches of row i are considered in order before considering a match of row $i + 1$). Such a preprocessing algorithm, referred to as “Algorithm Pre” in the rest of this paper, was presented in [13, 23], which runs in $O(\mathcal{R} \log \log n + n)$ time.⁵ Below, we briefly review Algorithm Pre for the sake of completeness.

We construct for each symbol $a \in \Sigma$ two separate lists, $L_X[a]$ and $L_Y[a]$. For each $a \in \Sigma$, $L_X[a]$ (resp. $L_Y[a]$) stores, in sorted order, the positions of a in X (resp. Y), if any. We now use an elegant data structure invented by van Emde Boas [26] that allows us to maintain a sorted list of integers in the range $[1..n]$ in $O(\log \log n)$ time per insertion and deletion. Additionally, it can return $Next(e)$ (successor element of

³In [13, 23], only the running time of LCS-I was mentioned without explicitly devising the algorithm. We revisit LCS-I here for the sake of completeness, because, LCS-II heavily depends on the concept used in LCS-I.

⁴Recall that, if $(i, j) \notin M$, $\mathcal{T}[i, j]$ is undefined.

⁵In [13, 23], the running time of Algorithm Pre is reported as $O(\mathcal{R} \log \log n)$ under the usual assumption that $\mathcal{R} \geq n$.

e in the list) and $Prev(e)$ (predecessor element of e in the list) in constant time. If $Prev(e)$ (resp. $Next(e)$) doesn't exist, then $null$ is returned. Using the lists, $L_X[a]$ and $L_Y[a]$ for all $a \in \Sigma$, we construct a van Emde Boas (vEB) data structure E_P , where we insert each pair (i, j) , $i \in L_X[a]$, $j \in L_Y[a]$, $a \in \Sigma$. In this case, the order of the elements in E_P is maintained according to the value $((i - 1) \times n + j)$. Note that, these values are within the range $[1..n^2]$ and hence the cost is $O(\log \log(n^2)) = O(\log \log n)$ per insertion and deletion. It is easy to verify that, using E_P , we can get all the pairs in the correct order to process them in row by row manner. The $2 \times |\Sigma|$ lists can be constructed in $O(n)$, by simply scanning X and Y in turn. Since there are in total $\mathcal{R}$ elements in M , the construction of E_P requires $O(\mathcal{R} \log \log n + n)$ time.

After we have computed the set M (using Algorithm Pre), we can start computing the entries $T[i, j]$, $(i, j) \in M$ according to (1). Since we are not calculating all the entries of the table, we need to use a global variable and appropriate pointers to keep track of the actual LCS. For the efficient implementation of the computation of (1), we utilize the following facts observed in [13, 23].

Fact 1 ([13, 23]) Suppose $(i, j) \in M$. Then for all $(i', j) \in M$, $i' > i$, (resp. $(i, j') \in M$, $j' > j$), we must have $T[i', j] \geq T[i, j]$ (resp. $T[i, j'] \geq T[i, j]$).

Fact 2 ([13, 23]) The calculation of the entry $T[i, j]$, $(i, j) \in M$, $1 \leq i, j \leq n$, is independent of any $T[\ell, q]$, $(\ell, q) \in M$, $\ell = i$, $1 \leq q \leq n$.

Following the techniques of [13, 23], we also use the following problem and relevant result.

Problem “RMAX” (Range Maxima Query Problem) We are given an array $A = a_1 a_2 \dots a_n$ of numbers. We need to preprocess A to answer the following form of queries:

Query Given an interval $I = [i_s..i_e]$, $1 \leq i_s \leq i_e \leq n$, the goal is to find the index k (or the value $A[k]$ itself) with maximum (minimum, in the case of Range Minima Query) value⁶ $A[k]$ for $k \in I$.

Theorem 1 ([4, 8]) Problem RMAX can be solved in $O(n)$ preprocessing time and $O(1)$ time per query.

The algorithm LCS-I proceeds as follows. We maintain an array H of length n , where, for the current value of $i \in [1..n]$ we have, $H[\ell] = \max_{1 \leq k < i, (k, \ell) \in M} (T[k, \ell])$, $1 \leq \ell \leq n$. The ‘max’ operation, here, returns 0, if there does not exist any $(k, \ell) \in M$ within the range. In other words, $H[\ell]$ stores the maximum value so far (at a particular stage of the computation) of column ℓ (see Fig. 1).

We use $RMAX_H(left, right)$ to denote the range maxima query on array H for the range $[left..right]$, $1 \leq left \leq right \leq n$. If $right < left$, then we assume that,

⁶Ties can be broken arbitrarily, e.g. by taking the one with larger (smaller) index.

Fig. 1 An example of Array H in a partially computed matrix according to (1). Computation is performed up to $T[4, 5]$. An $*$ denotes a matched entry

H	1	2	2	0	3	1
	1	2	3	4	5	6
C						*
C						*
A	*				*	
	1				3	
G		*	*			
		2	2			
A	*				*	
	1				1	
C						*
						1
	A	G	G	T	A	C

$RMAX_H(left, right)$ returns 0 as the index.⁷ Now, given the updated array H , we can easily perform the ‘max’ operation of (1) by using the constant time range maxima query (Theorem 1) to obtain $T[i, j] = RMAX_H(1, j - 1) + 1$. And, Fact 1 makes it easy to maintain the array H on the fly, as we proceed as follows. As usual, we proceed in a row by row manner. We use another array S , of length n , as a temporary storage. When we find an $(i, j) \in M$, after calculating $T[i, j]$, we store $S[j] = T[i, j]$. We continue to store in this way as long as we are in the same row. As soon as we find an $(i', j) \in M, i' > i$, i.e., we start processing a new row, we update H with new values from S . It is important to note that, due to Fact 1, we do not need to reset S after H is updated (by it) for the next row. The correctness of the above procedure follows from Facts 1 and 2. However, for the constant time range maxima query, we need to do an $O(n)$ preprocessing as soon as H is updated. But due to Fact 2, it is sufficient to perform this preprocessing once per row. So, the computational effort added for this preprocessing is $O(n^2)$ in total. Therefore, we get the following theorem.

Theorem 2 *LCS-I solves Problem LCS in $O(n^2 + \mathcal{R} \log \log n)$ time.*

One detail is that, for a match $(i, j) \in M$ such that $j = 1$, we need to evaluate $RMQ_H(1, 0)$. We handle this case as follows. We keep the index 0 of S (and hence of H) for this special case and we always have 0 stored at $S[0]$ ($H[0]$). Recall now that, by definition, $RMQ_H(1, 0)$ would return index 0, which is exactly what is needed to handle this case.

The outline of LCS-I is presented formally in the form of Algorithm 1. Since we are not calculating all the entries of the table, we, of course, need to use variables (*globalLCS.Instance* and *globalLCS.Value* in Algorithm 1) to keep track of $lcs(X, Y)$.

⁷This is required for the computation of $T[i, j]$, such that $j = 1$.

Algorithm 1 Outline of LCS-I

```

1: Construct the set  $M$  using Algorithm Pre. Let  $M_i = \{(i, j) \in M, 1 \leq j \leq n\}, 1 \leq i \leq n$ .
2:  $\text{globalLCS.Instance} = \epsilon$ 
3:  $\text{globalLCS.Value} = 0$ 
4: for  $i = 0$  to  $n$  {Index 0 is needed for the computation of  $T.\text{Value}[i, j], j = 1$ } do
5:    $S[i].\text{Value} = 0$  {Initialize the temporary array  $S$ }
6:    $S[i].\text{Instance} = \epsilon$ 
7: end for
8: for  $i = 1$  to  $n$  do
9:    $H = S$  {Update  $H$  for the next row}
10:  Preprocess  $H.\text{Value}$  for Range Maxima Query
11:  for each  $(i, j) \in M_i$  do
12:     $\text{maxindex} = \text{RMQ}_H(1, j - 1)$  {Range Maxima Query on Array  $H$ }
13:     $T.\text{Value}[i, j] = H[\text{maxindex}].\text{Value} + 1$ 
14:     $T.\text{Prev}[i, j] = H[\text{maxindex}].\text{Instance}$ 
15:     $S[j].\text{Value} = T.\text{Value}[i, j]$ 
16:     $S[j].\text{Instance} = (i, j)$ 
17:    if  $\text{globalLCS.Value} < T.\text{Value}[i, j]$  then
18:       $\text{globalLCS.Value} = T.\text{Value}[i, j]$ 
19:       $\text{globalLCS.Instance} = (i, j)$ 
20:    end if
21:  end for
22: end for
23: return  $\text{globalLCS}$ 

```

In particular, globalLCS.Value gives us $r(X, Y)$ and we can construct the $\text{lcs}(X, Y)$ (the reverse of $\text{lcs}(X, Y)$ to be precise) by starting from $\text{globalLCS.Instance}$ and then tracing back by using the Prev values of the table T .

Note that, we can shave off the $\log \log n$ term from the running time reported in Theorem 2 as follows. Since we have an n^2 term anyway in the running time, we do not need to compute the set M in the preprocessing step using Algorithm Pre. Instead, in Step 11, we consider (i, j) for all $j \in [1..n]$ and perform useful computation only when $(i, j) \in M$.

3 The Improved Algorithm

In this section, we present the main result of this paper. In particular, we improve the running time of LCS-I, as reported in Theorem 2, with some nontrivial modifications. The resulting Algorithm, LCS-II, will eventually run in $O(\mathcal{R} \log \log n + n)$ time. As is explained in the previous section, LCS-I exploits the constant time query operation (Theorem 1) of Problem RMAX. However, due to the $O(n)$ preprocessing step of RMAX, we can't eliminate the n^2 term from the running time of LCS-I. But a very important albeit easily observable fact is that, the range maxima queries made in LCS-I is always of a special form.

Fig. 2 Partial E_A with e_a, e_b , $\dots \rightarrow (\alpha_a, x_a) \rightarrow (\alpha_b, x_b) \rightarrow (\alpha_c, x_c) \rightarrow \dots$ and e_c

Fact 3 All the range maxima queries in Algorithm LCS-I are of the form $RMQ(1, j)$, $0 \leq j < n$.

From Fact 3, it seems that, Problem RMAX may be too general a tool to solve LCS and it seems to be worthwhile to look for a better solution exploiting the special query structure reported in Fact 3. Indeed, we shall show that, we can exploit this special structure in the query to avoid the $O(n)$ preprocessing step and hence the n^2 term from the running time reported in Theorem 2. However, the price we pay is that, the query time increases to $O(\log \log n)$. We present the idea as follows.

Assume that, we have an array $A[1..n]$, on which we want to apply the range maxima queries. We maintain a vEB data structure E_A , where each element $e_a \in E_A$, $1 \leq a \leq |E_A|$ is a 2-tuple $(Value, Pos)$. The order in E_A is maintained according to $e_a.Pos$, $1 \leq a \leq |E_A|$. Now, consider 3 entries $e_a, e_b, e_c \in E_A$ such that $e_b = Next(e_a)$ and $e_c = Next(e_b)$. Let $e_a \equiv (\alpha_a, x_a)$, $e_b \equiv (\alpha_b, x_b)$ and $e_c \equiv (\alpha_c, x_c)$ (Fig. 2). The invariants we maintain are as follows:

$$RMQ(1, x) = \alpha_a, \quad Prev(e_a).Pos < x \leq x_a, \quad (2)$$

$$RMQ(1, x) = \alpha_b, \quad x_a < x \leq x_b, \quad (3)$$

$$RMQ(1, x) = \alpha_c, \quad x_b < x \leq x_c. \quad (4)$$

Assuming that, we have the above data structure at our disposal, answering a query is easy as follows. Consider a query $RMQ(1, x')$. To answer this query, we just need to return the ‘Value’ of the element, which would be next in order, if a new element with $Pos = x'$ were inserted in E_A . So, we create an entry $e' \equiv (null, x')$ and insert it in E_A and get the ‘Value’ of the $Next(e')$. Finally, we delete e' from E_A . The only thing we need to ensure is that, if there is already an entry e in E_A such that $e.Pos = x'$, e' must be placed before e in E_A . This is to ensure that $Next(e') = e$, as required. This can be easily achieved, if we take ‘Value’ into account while preserving the order in E_A for equal values of ‘Pos’ and assume ‘null’ to be a lesser value than any other ‘Value’. Note however that, by definition, in ‘normal’ state, there can be no two elements in E_A having the same value for ‘Pos’. The steps are formally presented in Algorithm 2.

Algorithm 2 Function $Compute_RMQ(E_A, 1, x')$ (Steps to answer the query $RMQ(1, x')$ on E_A .)

- 1: Insert $e' \equiv (null, x')$ in E_A
 - 2: $Result = Next(e').Value$
 - 3: Delete e' from E_A
 - 4: **return** $Result$
-

Fig. 3 Initial E_H

$$(-\infty, -\infty) \rightarrow (0, j' - 1) \rightarrow (1, n) \rightarrow (\infty, \infty)$$

3.1 Update

Now, we discuss how we can maintain Invariant (2) to (4) under update operations in the context of the Algorithm LCS-I. Recall that, our goal is to get the answer of appropriate range maxima queries on the array H in Algorithm LCS-I and we operate in a row by row basis. In order to do that, we maintain a vEB structure E_H in light of the above discussion on E_A . For the sake of convenience, we use the following notation:

$$M_i = \{(i, j) \mid X[i] = Y[j], 1 \leq j \leq n\}.$$

We start with reporting the following fact.

Fact 4 *If $M_1 \neq \emptyset$, then for all $(i, j) \in M_1$, we have $T[i, j] = 1$.*

In cases where $M_1 = \emptyset$ or a number of consecutive $M_i = \emptyset, i \geq 1$, we have the following fact.

Fact 5 *$T[i, j] = 1$, for all $(i, j) \in M_i$ such that $M_i \neq \emptyset$ and $M_k = \emptyset$, for all $1 \leq k < i$.*

We initialize E_H with four elements, $e_{-\infty} \equiv (-\infty, -\infty)$, $e_s \equiv (0, j' - 1)$, $e_e \equiv (1, n)$ and $e_{\infty} \equiv (\infty, \infty)$, where $(1, j') \in M_1$ and there exists no $j < j'$ such that $(1, j) \in M_1$ (Fig. 3). Note that, if $M_1 = \emptyset$ then, for initialization, we have to use $M_i \neq \emptyset$ instead of M_1 such that $M_k = \emptyset$, for all $1 \leq k < i$ (Fact 5). This initialization of E_H , correctly maintains the invariants for the processing of the next row. Indeed, for the next row, we must have $RMQ(1, x) = 0$, if $x \leq j' - 1$ and $RMQ(1, x) = 1$ otherwise. The reason for keeping the first and the last element, namely, $e_{-\infty}$ and e_{∞} respectively, will be clear as we proceed. Here, we assume ∞ (resp. $-\infty$) to be greater (resp. smaller) than any number.

Now, let us consider the case where we process the subsequent rows. It is important to note that, as we process a particular row i , for each $(i, j) \in M_i$, we need to update E_H ; but this update is effective only for the next row, i.e. row $i + 1$. So, as we process row i , we perform the update on a temporary copy and as soon as row i is completely processed, we actually change the E_H to make it ready for row $i + 1$. In what follows, for the sake of convenience, we denote by E_H^i the ‘state’ of E_H which is used to process row i . Now, assume that, we are in row i and processing the match $(i, x' + 1) \in M_i$. So, we need the answer of the query $RMQ(1, x')$, which can be obtained easily applying Algorithm 2 on E_H^i . So, according to LCS-I, we would compute $T[i, x' + 1] = RMQ(1, x') + 1$. Assume that, $RMQ(1, x') + 1 = \alpha'$. Now, we need to consider the updating of E_H^i to get the E_H^{i+1} . We initialize E_H^{i+1} with E_H^i and for each match $(i, j) \in M_i$, we continue to update E_H^{i+1} so that we get the ‘correct’ E_H^{i+1} as soon as the processing of row i is finished. The update process is as follows. In what follows, we assume (without the loss of generality) that, we have $e_a, e_b, e_c \in E_H^{i+1}$ such that $e_b = \text{Next}(e_a)$, $e_c = \text{Next}(e_b)$ (Fig. 4). Assume, without

Fig. 4 Partial E_H^{i+1} with $\dots \rightarrow (\alpha_a, x_a) \rightarrow (\alpha_b, x_b) \rightarrow (\alpha_c, x_c) \rightarrow \dots$
 e_a, e_b , and e_c

Algorithm 3 Computation Row $i + 1$ of LCS-II

```

1:  $E_H^{i+1} = E_H^i$  {Initialization}
2: for all  $(i + 1, j) \in M_{i+1}$  do
3:    $x' = j - 1$ 
4:    $Result = Compute\_RMQ(E_H^i, 1, x')$  {Apply Algorithm 2. Note that, for the
     whole row  $i + 1$ , RMQ must be computed on  $E_H^i$ .}
5:    $\mathcal{T}[i, j] = result + 1$ 
6:    $\alpha' = \mathcal{T}[i, j]$ 
7:   Insert  $e' \equiv (null, x')$  in  $E_H^{i+1}$ 
8:    $e_a = e'.Prev$ 
9:    $e_b = e'.Next$ 
10:   $e_c = e_b.Next$ 
11:  Delete  $e'$  from  $E_H^{i+1}$ 
12:  {Steps 7 to 11 are performed to find  $e_a \equiv (\alpha_a, x_a), e_b \equiv (\alpha_b, x_b), e_c \equiv$ 
    $(\alpha_c, x_c) \in E_H^{i+1}$  such that  $e_b = Next(e_a), e_c = Next(e_b)$  (Fig. 4) and  $x_a < x' \leq$ 
    $x_b$ . Note that, this is similar to Algorithm 2; but thus must be done on  $E_H^{i+1}$  as
   opposed to on  $E_H^i$ , since there could be more than one match in row  $i + 1$ .}
13:  if  $j \leq x_b$  {Case 1:  $x' + 1 \leq x_b$ } then
14:    Insert  $e' \equiv (\alpha_b, x')$  in  $E_H^{i+1}$ 
15:    if  $\alpha_c = \alpha'$  {Case 1(a)} then
16:      Delete  $e_b$  from  $E_H^{i+1}$ 
17:    else if  $\alpha_c > \alpha'$  {Case 1(b)} then
18:      Delete  $e_b$  from  $E_H^{i+1}$ 
19:      Insert  $(\alpha', x_b)$  in  $E_H^{i+1}$  {Steps 18 and 19 actually perform the update on
        $e_b$ .}
20:    end if
21:  else if  $j = x_b + 1$  {Case 2:  $x' + 1 = x_b + 1$ } then
22:    Do Nothing
23:  end if
24: end for
25: return  $E_H^{i+1}$ 

```

Fig. 5 Updated E_H for Case 1.a $\dots \rightarrow (\alpha_a, x_a) \rightarrow (\alpha_b, x') \rightarrow (\alpha_c = \alpha', x_c) \rightarrow \dots$

the loss of generality that $x_a < x' \leq x_b$. Then, according to Invariants 2 to 4, we have $RMQ(1, x') = \alpha_b$. Therefore, $\alpha' = \alpha_b + 1$. Now, we have two cases.

Case 1: $x' + 1 \leq x_b$. In this case, we first insert a new element $e' \equiv (\alpha_b, x')$ to E_H^{i+1} .

Now, we have to change $e_b \equiv (\alpha_b, x_b)$. But this change may be influenced by $e_c \equiv (\alpha_c, x_c)$ as follows. We have two cases.

Case (a): $\alpha_c = \alpha' = \alpha_b + 1$. In this case, we just need to delete e_b (Fig. 5).

Fig. 6 Updated E_H for Case 1.b $\dots \rightarrow (\alpha_a, x_a) \rightarrow (\alpha_b, x') \rightarrow (\alpha', x_b) \rightarrow (\alpha_c, x_c) \rightarrow \dots$

Initialization (E_H^2): $(-\infty, -\infty) \rightarrow (0, 5) \rightarrow (1, 6) \rightarrow (\infty, \infty)$
 After Row 2 (E_H^3): $(-\infty, -\infty) \rightarrow (1, 6) \rightarrow (\infty, \infty)$
 After Row 3 (E_H^4): $(-\infty, -\infty) \rightarrow (1, 1) \rightarrow (2, 6) \rightarrow (\infty, \infty)$
 After Row 4 (E_H^5): $(-\infty, -\infty) \rightarrow (1, 1) \rightarrow (2, 4) \rightarrow (3, 6) \rightarrow (\infty, \infty)$

Fig. 7 The values of E_H from initialization up to the end of computation of Row 4 of the matrix of Fig. 1

E_H^5 : $(-\infty, -\infty) \rightarrow (1, 1) \rightarrow (2, 4) \rightarrow (3, 6) \rightarrow (\infty, \infty)$
 Step 1 $\Rightarrow (E_H^6)$: $(-\infty, -\infty) \rightarrow (1, 1) \rightarrow (2, 4) \rightarrow (3, 6) \rightarrow (\infty, \infty)$
 Step 3 $\Rightarrow x' = 6 - 1 = 5$
 Step 4 $\Rightarrow \text{Result} = 3$ Step 5 $\Rightarrow T[5, 6] = 3 + 1 = 4$
 Step 6 $\Rightarrow \alpha' = 4$ Step 7 to 11 $\Rightarrow e_a \equiv (2, 4), \quad e_b \equiv (3, 6), \quad e_c \equiv (\infty, \infty)$
 Step 14 $\Rightarrow (E_H^6)$: $(-\infty, -\infty) \rightarrow (1, 1) \rightarrow \begin{array}{c} (2, 4) \nearrow (3, 6) \\ \searrow \quad \nearrow \\ (3, 5) \end{array} \rightarrow (\infty, \infty)$
 Step 18 and 19 $\Rightarrow (E_H^6)$: $(-\infty, -\infty) \rightarrow (1, 1) \rightarrow \begin{array}{c} (2, 4) \nearrow (3, 6) \\ \searrow \quad \nearrow \\ (3, 5) \end{array} \rightarrow (\infty, \infty)$
 Step 25 $\Rightarrow \text{Return } E_H^6$: $(-\infty, -\infty) \rightarrow (1, 1) \rightarrow (2, 4) \rightarrow (3, 5) \rightarrow (4, 6) \rightarrow (\infty, \infty)$

Fig. 8 Execution of Algorithm 3 while computing E_H^6 from E_H^5 of Example 1

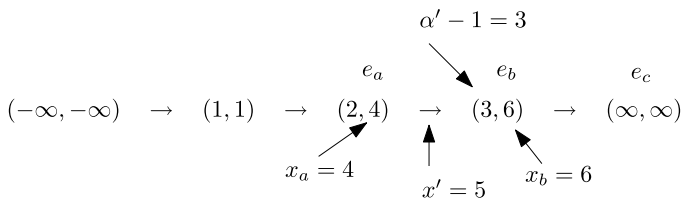


Fig. 9 Relation among $x', e_a, e_b, e_c, \alpha'$ while computing E_H^6 from E_H^5 of Example 1

Case (b): $\alpha_c > \alpha' = \alpha_b + 1$. In this case, we update e_b such that $e_b.value = \alpha'$ (Fig. 6).

Case 2: $x' + 1 = x_b + 1$. In this case, we don't do anything.

The steps for the computation of Row $i + 1$ is presented formally in the form of Algorithm 3. Below, we present an example for better understanding.

Example 1 Consider the partially computed matrix of Fig. 1. The states of E_H after the computation of Row 1 (i.e. initialization) up to Row 4 are presented in Fig. 7. The computation of $T[5, 6]$ (using E_H^5) and that of E_H^6 , according to Algorithm 3, are shown in Fig. 8. The relation among $x', e_a, e_b, e_c, \alpha'$ is shown in Fig. 9.

In the following section, we present the formal proof of correctness of LCS-II. We also analyze its running time. In what follows, the above data structure is referred to as DS_RMQ.

3.2 Correctness and Running Time

In this section, we formally prove the correctness and deduce the running time of our algorithm. We first present the following useful lemmas.

Lemma 1 *In DS_RMQ, we always have $|E_H| > 3$.*

Proof During initialization we have $|E_H| > 3$ and during update, every delete is counter-balanced by an insert. Therefore, the lemma follows. $\square$

Lemma 2 *In DS_RMQ, we always have $e_{-\infty}, e_{\infty} \in E_H$, where $e_{-\infty} \equiv (-\infty, -\infty)$ and $e_{\infty} \equiv (\infty, \infty)$.*

Proof During initialization we have $e_{-\infty}, e_{\infty} \in E_H$. And, it is easy to realize that, during update, we never delete $e_{-\infty}$ or e_{∞} . Therefore, the lemma follows. $\square$

Lemma 3 *In DS_RMQ, the followings hold true:*

- A. *There can be no $e_a, e_b \in E_H, i \neq j$, such that $e_a.Pos = e_b.Pos$.*
- B. *There can be no $e_a, e_b \in E_H, i \neq j$, such that $e_a.Value = e_b.Value$.*

Proof It is easy to verify that, the lemma is true for the initial E_H . So, we now concentrate on update. During update, we either do nothing (Case 3.1) or insert a new element followed by either a delete or an update of ‘Value’ of an existing element (Case 3.1). So, we only need to consider the insert of Case 3.1. Now, in E_H , before the insertion, we have, $Next(e_a) = e_b$ such that $e_a \equiv (x_a, \alpha_a)$, $e_b \equiv (x_b, \alpha_b)$. We also have $x' > x_a$ and $x' + 1 \leq x_b \Leftrightarrow x' < x_b$. Now, Statement A is true because the inserted element e' is such that $e'.Pos = x'$. Statement B is true as follows. Recall that, we insert e' such that $e'.Value = \alpha_b (= e_b.Value)$. However, this insert is followed by either the deletion of e_b or the update of $e_b.Value$ from α_b to α' . Therefore, Statement B follows. $\square$

Remark 1 Note that, during the query procedure an ‘artificial’ element is inserted in E_H , when we may get two elements having the same Pos . But recall that, this ‘artificial’ element is deleted at the end of the query procedure and hence Lemma 3 remains valid under the ‘normal’ state of E_H .

Lemma 4 *If E_H in DS_RMQ has an element e , such that $e.Pos = x'$, then we must have $RMQ(1, x') = e.Value$.*

Proof Directly follows from Lemma 3 and Invariants 2 to 4. $\square$

Lemma 5 *Assume that there exists no $e \in E_H$ in DS_RMQ such that $e.Pos = x'$. Then, we have $RMQ(1, x') = e'.Value$, where $e'.Pos > x'$ and there exists no $e \in E_H$ such that $x' < e.Pos < e'.Pos$.*

Proof Directly follows from Lemma 3 and Invariants 2 to 4. $\square$

In the lemmas and theorems that follow, in line of our update procedure presented in Sect. 3.1, we assume the following scenario (without the loss of generality):

We assume that, we have just computed $T[i, x' + 1]$ and that $T[i, x' + 1] = \alpha'$.

We also assume that, we have $e_a, e_b, e_c \in E_H^{i+1}$ such that $e_b = \text{Next}(e_a)$, $e_c = \text{Next}(e_b)$. We further assume that, $x_a < x' \leq x_b$.

From Lemmas 1 and 2, it follows that, we can assume the above scenario without the loss of generality. Now we have the following lemmas.

Lemma 6 *We have $\alpha' = \alpha_b + 1$.*

Proof Since $x_a < x' \leq x_b$, according to Invariants 2 to 4, we have $\text{RMQ}(1, x') = \alpha_b$. Therefore, $\alpha' = \text{RMQ}(1, x') + 1 = \alpha_b + 1$. $\square$

Lemma 7 *In DS_RMQ, there is always an element $e_n \in E_H$ such that $e_n.\text{Pos} = n$.*

Proof Recall that, the initial E_H has an element $(1, n)$. Now, during the update, we either do nothing (Case 3.1) or insert a new element followed by, either a delete or an update of ‘Value’ of an existing element (Case 3.1). Now, we only need to focus on delete, which is handled in Case 3.1. In this case, we have $\alpha_c = \alpha' = \alpha_b + 1$. Now, recall that, in this case, we delete e_b . So, it suffices to show that $e_b.\text{Pos} \neq n$. Assume for the sake of contradiction that $e_b.\text{Pos} = n$. Then, we must have $e_c = e_\infty$. This follows from the fact that, all the queries are of the form $\text{RMQ}(1, x)$, where $x < n$ and hence we never need to insert any element with $\text{Pos} = n + 1$. Therefore, we have $\alpha_c = \infty$, a contradiction (since $\alpha_c = \alpha_b + 1$). So, this means that we can never have $e_b.\text{Pos} = n$ and hence the result follows. $\square$

The following theorem proves the correctness of the query procedure (Algorithm 2) and deduce the corresponding running time.

Theorem 3 *Given the data structure DS_RMQ, Algorithm 2 correctly answers the $\text{RMQ}(1, x')$, $1 \leq x' < n$ in $O(\log \log n)$ time per query.*⁸

Proof We first discuss the correctness and then deduce the running time.

Correctness: There can be two cases.

Case 1: There is an element $e \in E_H$ such that $e.\text{Pos} = x'$. Then, according to Lemma 4, we have $\text{RMQ}(1, x') = e.\text{Value}$. Now, Algorithm 2 inserts the element $e' \equiv (\text{null}, x')$ in E_H . Since we have assumed that ‘null’ is lesser than any other ‘Value’, after this insertion, we will have $\text{Next}(e') = e$. Therefore, Algorithm 2 will return $e.\text{Value}$ as the result which is correct (Lemma 4).

⁸Note that, for LCS computation we never require to check $\text{RMQ}(1, x')$, where $x' \geq n$.

Case 2: There is no element in E_H having $Pos = x'$. Note that, $x' < n$. Now, according to Lemma 7, we always have an element $e_n \in E_H$ such that $e_n.Pos = n$. Therefore, we always have an element other than e_∞ which has Pos greater than x' . Consider $e' \in E_H$ such that $e'.Pos > x'$ and there exists no $e \in E_H$ such that $x' < e.Pos < e'.Pos$. Then, according to Lemma 5, $RMQ(1, x') = e'.Value$. Now, Algorithm 2 inserts the element $e'' \equiv (null, x')$ in E_H . Therefore, after this insertion, we will have $Next(e'') = e'$. Hence, Algorithm 2 will return $e'.Value$ as the result which is correct (Lemma 5).

Running Time: In Algorithm 2, we perform one insert, one delete and one constant time query for next value in the underlying vEB structure. Now the highest value of the ‘Value’ field of any element can’t exceed n , because $r(X, Y) \leq n$. Therefore, both insert and delete requires $O(\log \log n)$ time each. Hence, the total running time of Algorithm 2 is $O(\log \log n)$. $\square$

Next, we prove the correctness of the update procedure presented in Sect. 3.1. First, we present the following lemma.

Lemma 8 *In DS_RMQ , for each pair of elements $e_a, e_b \in E_H$, such that $Next(e_a) = e_b$ and $e_a, e_b \notin \{e_{-\infty}, e_\infty\}$, we have $e_b.Value = e_a.Value + 1$.*

Before proving Lemma 8, we first need to present and prove the following claims.

Claim 1 *If Lemma 8 is valid beforehand, it remains valid after Case 1a is executed.*

Proof Assume that Lemma 8 is valid beforehand. Now, in Case 1a, we first insert a new element $e' \equiv (\alpha_b, x')$ to E_H^{i+1} and then delete $e_b \equiv (\alpha_b, x_b)$. So it suffices to show that $e'.Value = \alpha_b = e_a.Value + 1$ and $e_c.Value = e'.Value + 1 (= \alpha_b + 1)$. This follows immediately because, before the update, due to Lemma 8, we had $e_b.Value = \alpha_b = e_a.Value + 1$ and $e_c.Value = e_b.Value + 1 = \alpha_b + 1$. $\square$

Claim 2 *If Lemma 8 is valid before Case 1b arises, then, in Case 1b, we must have $e_c = e_\infty$.*

Proof Assume that Lemma 8 is valid. Now, in Case 1b, we have $\alpha_c > \alpha' = \alpha_b + 1$. Therefore, it is clear that we must have $e_c = e_\infty$, because otherwise, according to Lemma 8, we would have $e_c.Value = e_b.Value + 1 \Leftrightarrow \alpha_c = \alpha_b + 1$, a contradiction. $\square$

Claim 3 *If Lemma 8 is valid before Case 1b arises, then, after the execution of Case 1b, Lemma 8 still remains valid.*

Proof Assume that, Lemma 8 is valid beforehand. Here, we first insert a new element $e' \equiv (\alpha_b, x')$ to E_H^{i+1} . Then, we update e_b such that $e_b.value = \alpha'$. Let us denote by e_b^{before} and e_b^{after} , the states of e_b before and after the update respectively. Then, it suffices to show that $e'.Value = e_a.Value + 1$ and $e_b^{after}.Value = e'.Value + 1$ (we

don't need to show the same relation between $e_c.Value$ and $e_b^{after}.Value$ because $e_c \in \{e_{-\infty}, e_{\infty}\}$ (Claim 2)). We show it as follows. Before update, we have $e_b^{before}.Value = \alpha_b = e_a.Value + 1$ and $e_c.Value = e_b^{before}.Value + 1 = \alpha_b + 1$. Since $e'.Value = \alpha_b$, we have $e'.Value = e_a.Value + 1$. Finally, recall that, $\alpha' = \alpha_b + 1$. Therefore, since $e_b^a.Value = \alpha'$, we have $e_b^{after}.Value = e'.Value + 1$. $\square$

Now we are ready to prove Lemma 8.

Proof of Lemma 8 The lemma is true for the initial E_H . So, it suffices to show that, it remains valid under the updates. Now, during update, we do useful work only in Case 1 and we have two subcases under this case, namely Case 1a and 1b. Since the lemma is true for the initial E_H , by Claim 1, we have that, if Case 1b never arises, the lemma remains valid. Therefore, when Case 1b arises for the very first time, this lemma remains valid beforehand and hence by Claim 3, it still remains valid after the execution of Case 1b. Therefore, the result follows. $\square$

Theorem 4 *The procedure in Sect. 3.1 on DS_RMQ is correct.*

Proof It is easy to verify that, the initialization of E_H is correct. Now, suppose that, we are considering the match $(i, x' + 1) \in M_i$. Therefore, we require the answer of the query $RMQ(1, x')$, which can be correctly obtained using Algorithm 2 on E_H^i (Theorem 3). Now, we calculate $T[i, x' + 1] = RMQ(1, x') + 1$. Assume that $RMQ(1, x') + 1 = \alpha'$. This means that, we now have, at position $x' + 1$, a new value α' . By Lemma 6, we have that $\alpha' = \alpha_b + 1$. Now, it suffices to show that, the update procedure correctly updates E_H^{i+1} to reflect this new situation. According to the procedure in Sect. 3.1, we now have two cases as discussed below.

We consider Case 2 first, because, it is simpler than the other one. In Case 2, we have $x' + 1 = x_b + 1$ and we do nothing. This is correct as follows. In this case, we have the value $\alpha_b + 1$ at position $x' + 1$. Now, since $x' < n$, we have $x' + 1 \leq n$. Since there is always an element in E_H with $Pos = n$ (Lemma 7), we have $e_c \neq e_{\infty}$. Hence, by Lemma 8, we have $\alpha_c = \alpha_b + 1$. This means that, the value $\alpha_b + 1$ at position $x' + 1$ is already covered by e_c and we need not do anything.

Now we consider the Case 1, where we have $x' + 1 \leq x_b$. Recall that, $x' > x_a$. Here we have two subcases depending on e_c . In Case 1a, we have $\alpha_c = \alpha'$ and in Case 1b, we have $\alpha_c > \alpha'$. Considering these two (sub)cases suffices, because, $\alpha_c > \alpha_b$ (Lemma 8) and we have $\alpha' = \alpha_b + 1$ (Lemma 6). We now consider each of the subcases separately below.

First, consider Case 1a. According to our procedure, we first insert a new element $e' \equiv (\alpha_b, x')$ to E_H^{i+1} and then delete $e_b = (\alpha_b, x_b)$. The insertion is correct, because, we have a (new) value $\alpha' = \alpha_b + 1$ at position $x' + 1 \leq x_b$, which means that, up to $x' < x_b$ we have no change in the RMQ answers. Now, recall that, in this case, we have $\alpha_c = \alpha'$. Therefore, the new value is already covered by e_c and hence the subsequent deletion of e_b from E_H^{i+1} is correct as well.

Now, we consider Case 1b. Here as well, we first insert a new element $e' \equiv (\alpha_b, x')$ to E_H^{i+1} . Then we update e_b such that $e_b.value = \alpha'$. The insert is correct as argued above (in Case 1a). Now, in this case, we have $\alpha_c > \alpha' = \alpha_b + 1$. Therefore, by

Lemma 8 and Claim 2, we must have $e_c = e_\infty$. And, it is clear that, up to position x_b , we have α' as the highest value and hence, the update of e_b such that $e_b.\text{Value} = \alpha'$ is correct.

This completes the proof of the theorem. $\square$

Now we analyze the running time of Algorithm LCS-II.

Theorem 5 *LCS-II solves Problem LCS in $O(\mathcal{R} \log \log n + n)$ time.*

Proof Firstly, we use Algorithm Pre to pre-compute the set M in the prescribed order requiring $O(\mathcal{R} \log \log n + n)$ time. Then, for each $(i, j) \in M$, we do the following 3 steps.

- 1: Perform appropriate range maxima query using Algorithm 2.
- 2: Compute $T[i, j]$ from the result of Step 1.
- 3: Update E_H using Algorithm 3.

Step 1, i.e., Algorithm 2 requires $O(\log \log n)$ time (Theorem 3). Step 2 requires $O(1)$ time. In Step 3, we perform the update. Here, we either do nothing or insert a new element followed by, either a delete or an update of ‘Value’ of an existing element. The latter operation, i.e. update, is basically done by a delete (Step 18 of Algorithm 3) followed by an insert with the changed ‘Value’ and same ‘Pos’ (Step 19 of Algorithm 3). Additionally, we perform one extra insert and update each to locate the corresponding e_a , e_b and e_c (Step 7 and 11 of Algorithm 3, respectively). Therefore, in the worst case, we perform three inserts and two deletes in the underlying vEB structure. Now, the highest value of the ‘Value’ field of any element can’t exceed n , because $r(X, Y) \leq n$. So, both insert and delete requires $O(\log \log n)$ time each and hence, Step 3.2 requires $O(\log \log n)$ time in the worst case. Therefore, Algorithm LCS-II requires $O(\mathcal{R} \log \log n + n)$ time to compute LCS. $\square$

4 Conclusion

In this paper, we have studied the classic and much studied LCS problem for two strings. Using some interesting techniques of [23], and combining them with a new data structure to solve a restricted dynamic version of the Range Maxima Query problem, we have presented an $O(\mathcal{R} \log \log n + n)$ time algorithm to compute the LCS, where $\mathcal{R}$ is the total number of ordered pairs of positions at which the two strings match. Although $\mathcal{R} = O(n^2)$, there are large number of applications for which we have $\mathcal{R} \sim n$. Typical of such applications include finding the longest ascending subsequence of a permutation of integers from 1 to n , finding a maximum cardinality linearly ordered subset of some finite collection of vectors in 2-space etc (for more details see [11] and references therein). So in these situations, our algorithm would exhibit an almost linear $O(n \log \log n)$ behavior. The techniques we have used to develop our algorithm are new and, we believe, of independent interest. We believe a number of interesting issues remain as candidates for future research as follows.

- A. The LCS problem for more than two strings along with other variants have extensive applications e.g. in molecular biology. It would be interesting to see whether our techniques can be extended for LCS problems involving more than two strings or variants thereof (e.g. constrained LCS [2, 3, 25], rigid LCS [17]).
- B. We have implicitly presented an algorithm for the range maxima query problem. Our algorithm allows restricted but dynamic updates and considers a restricted sets of queries. It would be interesting to see whether we can lift the restrictions and/or improve the query and pre-processing time.
- C. Finally, it would be interesting to see how LCS-II performs in practice and to compare its performance with other LCS algorithms, especially with the celebrated algorithm of Hunt and Szymanski [11].

Acknowledgements The authors would like to express their gratitude to the anonymous reviewers for their helpful suggestions and comments.

References

1. Arlazarov, V.L., Dinic, E.A., Kronrod, M.A., Faradzev, I.A.: On economic construction of the transitive closure of a directed graph (English translation). *Sov. Math. Dokl.* **11**, 1209–1210 (1975)
2. Arslan, A.N., Eggecioglu, Ö: Algorithms for the constrained longest common subsequence problems. In: *Stringology*, pp. 24–32 (2004)
3. Arslan, A.N., Eggecioglu, Ö: Algorithms for the constrained longest common subsequence problems. *Int. J. Found. Comput. Sci.* **16**(6), 1099–1109 (2005)
4. Bender, M.A., Farach-Colton, M.: The lca problem revisited. In: *LATIN*, pp. 88–94 (2000)
5. Bergroth, L., Hakonen, H., Raita, T.: A survey of longest common subsequence algorithms. In: *SPIRE*, pp. 39–48 (2000)
6. Støltting Brodal, G., Kaligosi, K., Katriel, I., Kutz, M.: Faster algorithms for computing longest common increasing subsequences, pp. 330–341 (2006)
7. Crochemore, M., Landau, G.M., Ziv-Ukelson, M.: A sub-quadratic sequence alignment algorithm for unrestricted cost matrices. In: *Symposium of Discrete Algorithms (SODA)*, pp. 679–688 (2002)
8. Gabow, H., Bentley, J., Tarjan, R.: Scaling and related techniques for geometry problems. In: *STOC*, pp. 135–143 (1984)
9. Hadlock, F.: Minimum detour methods for string or sequence comparison. *Congr. Numer.* **61**, 263–274 (1988)
10. Hirschberg, D.S.: Algorithms for the longest common subsequence problem. *J. ACM* **24**(4), 664–675 (1977)
11. Hunt, J.W., Szymanski, T.G.: A fast algorithm for computing longest subsequences. *Commun. ACM* **20**(5), 350–353 (1977)
12. Iliopoulos, C.S., Rahman, M.S.: New efficient algorithms for the lcs and constrained lcs problems. *Inf. Process. Lett.* (to appear). doi: [10.1016/j.ipl.2007.09.008](https://doi.org/10.1016/j.ipl.2007.09.008)
13. Iliopoulos, C.S., Rahman, M.S.: Algorithms for computing variants of the longest common subsequence problem. *Theor. Comput. Sci.* (2007, to appear at a special issue)
14. Iliopoulos, C.S., Rahman, M.S.: New efficient algorithms for LCS and constrained LCS problem. In: Broersma, H., Dantchev, S.S., Johnson, M., Szeider, S. (eds.) *ACiD, Texts in Algorithmics*, vol. 9, pp. 83–94. King's College, London (2007)
15. Jiang, T., Li, M.: On the approximation of shortest common supersequences and longest common subsequences. *SIAM J. Comput.* **24**(5), 1122–1139 (1995)
16. Levenshtein, V.I.: Binary codes capable of correcting deletions, insertions, and reversals. *Probl. Inf. Transm.* **1**, 8–17 (1965)
17. Ma, B., Zhang, K.: On the longest common rigid subsequence problem. In: *CPM*, pp. 11–20 (2005)
18. Maier, D.: The complexity of some problems on subsequences and supersequences. *J. ACM* **25**(2), 322–336 (1978)
19. Masek, W.J., Paterson, M.: A faster algorithm computing string edit distances. *J. Comput. Syst. Sci.* **20**(1), 18–31 (1980)

20. Mäkinen, V., Navarro, G., Ukkonen, E.: Transposition invariant string matching. *J. Algorithms* **56**, 124–153 (2005)
21. Myers, E.W.: An $o(nd)$ difference algorithm and its variations. *Algorithmica* **1**(2), 251–266 (1986)
22. Nakatsu, N., Kambayashi, Y., Yajima, S.: A longest common subsequence algorithm suitable for similar text strings. *Acta Inf.* **18**, 171–179 (1982)
23. Rahman, M.S., Iliopoulos, C.S.: Algorithms for computing variants of the longest common subsequence problem. In: Asano, T. (ed.) ISAAC. *Lecture Notes in Computer Science*, vol. 4288, pp. 399–408. Springer, Berlin (2006)
24. Rahman, M.S., Iliopoulos, C.S.: A new efficient algorithm for computing the longest common subsequence. In: Kao, M.-Y., Li, X.-Y. (eds.) AAIM. *Lecture Notes in Computer Science*, vol. 4508, pp. 82–90. Springer, Berlin (2007)
25. Tsai, Y.-T.: The constrained longest common subsequence problem. *Inf. Process. Lett.* **88**(4), 173–176 (2003)
26. van Emde Boas, P.: Preserving order in a forest in less than logarithmic time and linear space. *Inf. Process. Lett.* **6**, 80–82 (1977)
27. Wagner, R.A., Fischer, M.J.: The string-to-string correction problem. *J. ACM* **21**(1), 168–173 (1974)